
Paradigmas de Programación

— —

CYPLP

Paradigmas

Un paradigma de programación es un **estilo de desarrollo de programas**, un modelo para resolver problemas computacionales. Los lenguajes de programación, necesariamente, se encuadran en uno o varios paradigmas a la vez, a partir del tipo de órdenes que permiten implementar, tiene una **relación directa con su sintaxis**.

```
1 procedure DandC(pbm, sol)
2 local var aux;
3 begin
4   if easy(pbm) then
5     solve(pbm)
6   else
7     begin
8       divide(pbm, subpbm, aux);
9       parallel i in 1..numP
10         DandC(subpbm[i], subsol[i]);
11       combine(subsol, aux, sol);
12     end
13 end;
```



```
?- p_exam(noest,notar).
false.
```

```
?- p_exam(est,tar).
true .
```

```
?- p_exam(noest,tar).
true.
```

```
fib :: Integer -> Integer
fib 0 = 0
fib 1 = 1
fib n = fib (n-1) + fib (n-2)
```

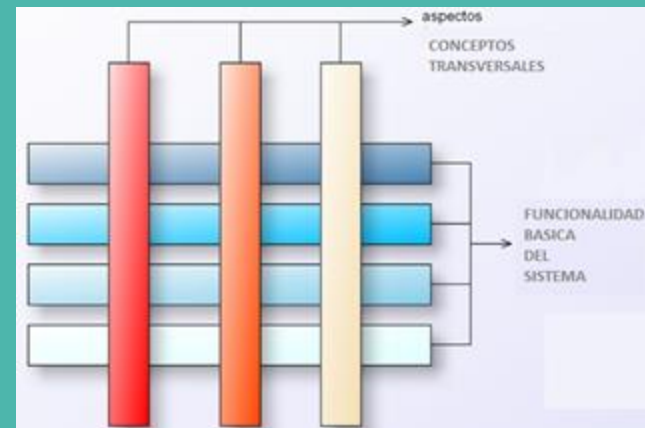
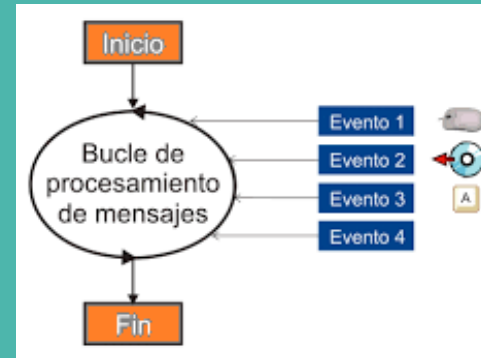
Principales Paradigmas

- **Imperativo:** sentencias + secuencias de comandos
- **Declarativo.** Los programas describen los resultados esperados sin listar explícitamente los pasos a llevar a cabo para alcanzarlos.
 - **Lógico.** Aserciones lógicas: hechos + reglas, es declarativo
- **Funcional.** Los programas se componen de funciones
- **Orientado a Objetos :** Métodos + mensajes.

Principales Paradigmas

Otra forma de clasificación más reciente:

- Dirigido por eventos. El flujo del programa está determinado por sucesos externos (por ejemplo, una acción del usuario).
- Orientado a aspectos. Apunta a dividir el programa en módulos independientes, cada uno con un [Video explicativo](#). comportamiento y responsabilidad bien definido.
- Programación Reactiva
Relacionada con el tratamiento de flujo de datos ya que en principio busca reaccionar ante un evento de forma asincrónica.



Programación Orientada a Objetos

Programación Orientada a Objetos

“Un programa escrito con una lenguaje OO es un conjunto de OBJETOS que **INTERACTÚAN** mandándose **MENSAJES**”

Los elementos que intervienen en la programación OO son:

- Objetos
- Mensajes
- Métodos
- Clases

Objetos:

- Son entidades que poseen estado interno y comportamiento
- Es el equivalente a un dato abstracto

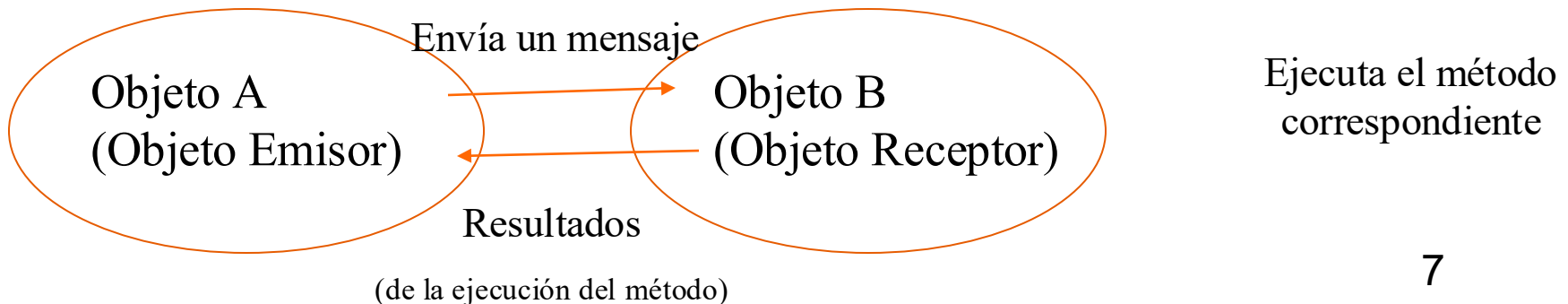
Programación Orientada a Objetos

Mensajes:

- Es una petición de un objeto a otro para que este se comporte de una determinada manera, ejecutando uno de sus métodos
- TODO el procesamiento en este modelo es activado por mensajes entre objetos.

Métodos:

- Es un programa que está asociado a un objeto determinado y cuya ejecución solo puede desencadenarse a través de un mensaje recibido por éste o por sus descendientes



Programación Orientada a Objetos

Clases:

- Es un tipo definido por el usuario que determina las estructuras de datos y las operaciones asociadas con ese tipo
- Cada objeto pertenece a una clase y recibe de ella su funcionalidad
- Primer nivel de abstracción de datos: definimos estructura, comportamiento y tenemos ocultamiento.
- La información contenida en el objeto solo puede ser accedida por la ejecución de los métodos correspondientes

Instancia de clase:

- Cada vez que se construye un objeto se está creando una INSTANCIA de esa clase
- Una instancia es un objeto individualizado por los valores que tomen sus atributos

Programación Orientada a Objetos

Otro aspecto de las abstracciones de datos

GENERALIZACIÓN/ESPECIFICACIÓN



HERENCIA

El segundo nivel de abstracción consiste en agrupar las clases en jerarquías de clases (definiendo SUB y SUPER clases), de forma tal que una clase A herede todas las propiedades de su superclase B (suponiendo que tiene una)

Ejemplo: Se tiene definido la siguiente clase

PERSONA

- Nombre
- Edad
- Sexo
- Documento
- Dirección
- Teléfono

- ver-nombre**
- ver-edad**
- ver-teléfono**
- ver-documento**
- ver-sexo**
- cambiar-dirección**
- sacar-documento, etc.**

EMPLEADO

- Curriculum
- cuil

Programación Orientada a Objetos

Otros conceptos adicionales

Polimorfismo:

- Es la capacidad que tienen los objetos de distintas clases de responder a mensajes con el mismo nombre

Ejemplo:

$3 + 5$ Se aplica suma entre
números

“Buenos” + “días” Se concatenan
strings

Binding dinámico:

Es la vinculación en el proceso de ejecución de los
objetos con los mensajes

Programación Orientada a Objetos

C++ (Lenguaje híbrido) Algunas características

- Lenguaje extendido del lenguaje C
- Incorporó características de POO

Los objetos en C++:

- Se agrupan en tipos denominados clases
- Contienen datos internos que definen su estado interno
- Soportan ocultamiento de datos
- Los métodos son los que definen su comportamiento
- Pueden heredar propiedades de otros objetos
- Pueden comunicarse con otros objetos enviándose mensajes

Programación Lógica

Programación Lógica

- La programación lógica es un tipo de paradigmas de programación dentro del paradigma de programación declarativa.
- Es un paradigma en el cual los programas son una serie de aserciones lógicas.
- El conocimiento se representa a través de reglas y hechos.
- Los objetos son representados por términos, los cuales contienen constantes y variables.
- PROLOG es el lenguaje lógico más utilizado.

Elementos de la Programación Lógica

La sintaxis básica es el “**término**”.

- **Variables:**

Se refieren a elementos indeterminados que pueden sustituirse por cualquier otro.

- “*humano(X)*”, la X puede ser sustituida por constantes como: juan, pepe, etc.
- Los nombres de las variables comienzan con mayúsculas y pueden incluir números.

- **Constantes:**

A diferencia de las variables son elementos determinados.

“*humano(juan)*”

Las constantes son string de letras en minúsculas (representan objetos atómicos) o string de dígitos (representan números).

Elementos de la Programación Lógica

Término compuesto:

- Consisten en un "functor" seguido de un número fijo de argumentos encerrados entre paréntesis, los cuales son a su vez términos.
- Se denomina "aridad" al número de argumentos.
- Se denomina "estructura" (ground term) a un término compuesto ***cuyos argumentos no son variables.***

Ejemplos:

padre	→	constante
Longitud	→	variable
tamaño(4,5)	→	estructura

Un functor es una estructura sintáctica utilizada para agrupar términos y formar abstracciones más complejas o estructuradas.

Elementos de la programación Lógica

Listas:

- La constante `[]` representa una lista vacía
- El functor “.” construye una lista de un elemento y una lista. Ejemplo: `.(alpha,[])`, representa una lista que contiene un único elemento que es `alpha`.
- Otra manera de representar la lista es usando `[]` en lugar de `.( )`. Ejemplo anterior la lista quedaría: `[alpha,[]]`
- Y también se representa utilizando el símbolo `|`
`[alpha | []]`

La notación general para denotar lista es : `[X | Y]`
`X` es el elemento cabeza de la lista e `Y` es una lista, que representa la cola de la lista que se está modelando

Cláusula de Horn

- Un programa escrito en un lenguaje lógico es una secuencia de “cláusulas”.
- Las cláusulas pueden ser: un “Hecho” o una “Regla”.

Hecho:

- Expresan relaciones entre objetos
- Expresan verdades
- Son expresiones del tipo $p(t_1, t_2, \dots, t_n)$

Ejemplos:

- $\text{tiene}(\text{coche}, \text{ruedas})$ $\square$ representa el hecho que un coche tiene ruedas
- $\text{longitud}([], 0)$ $\square$ representa el hecho que una lista vacía tiene longitud cero
- $\text{moneda}(\text{peso})$ $\square$ representa el hecho que peso es una moneda.

Cláusula de Horn

Regla:

- Cláusula de Horn
- Tiene la forma: conclusión :- condición. (‘:-’ sintaxis prolog)

Dónde:

- :- indica “Si”
 - conclusión es un simple predicado y
 - condición es una conjunción de predicados, separados por comas. Representan un AND lógico
-
- En un lenguaje procedural una regla la podríamos representar como: if condición else conclusión.

Programas y Queries

Ejemplo de programa:

OR { longitud ([],0). **HECHO**
longitud ([X | Y],N) :- longitud(Y, M), N $\neq$ M + 1. **AND**

Programa: conjunto de cláusulas **REGLA**

?-longitud([rojo | [verde | [azul | []]]],X).

Query: Representa lo que deseamos que sea contestado **QUERY**

Programas y Queries

Programa:

longitud ([],0).

longitud ([X | Y],N) :- longitud(Y, M), N=M + 1.

?-longitud([rojo | [verde | [azul | []]]],X).

longitud([],T) T=0

longitud([azul | []],Z) Z=T+1 => Z=1

longitud([verde | [azul | []]],M) M=Z+1 => M=2

longitud([rojo | [verde | [azul | []]]],X) X=M+1 => X=3

Ejecución de Programas

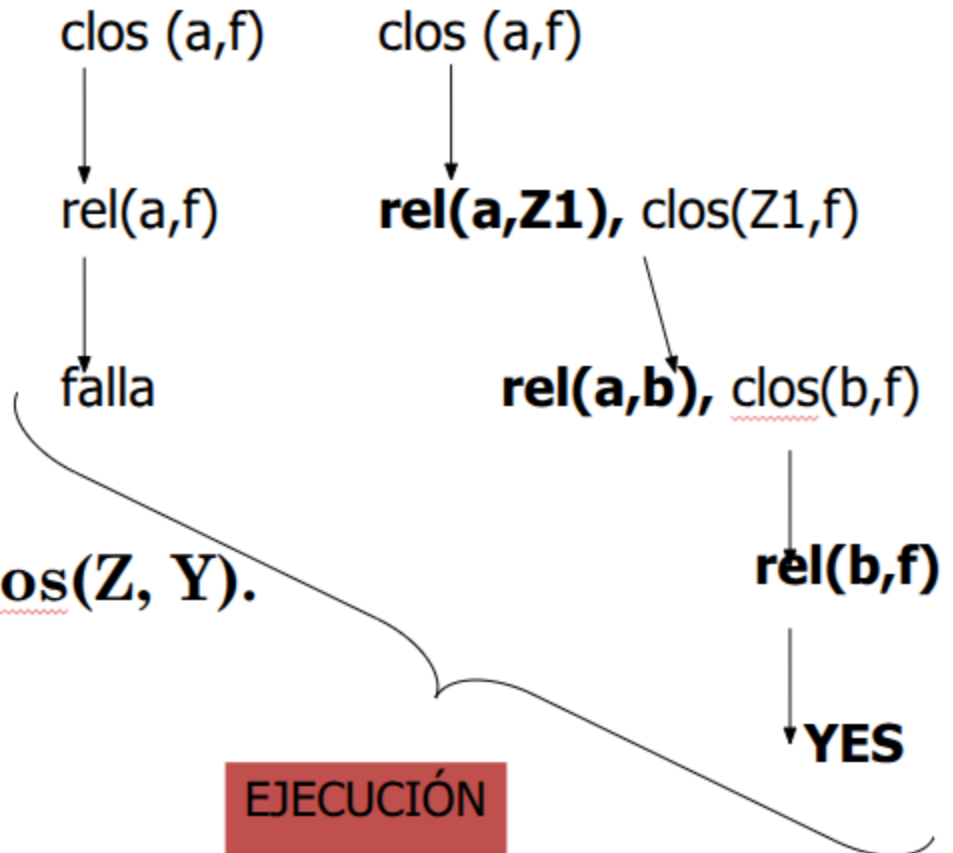
- Un programa es un conjunto de reglas y hechos que proveen una especificación declarativa de que es lo que se conoce y la pregunta es el objetivo que queremos alcanzar.
- La ejecución de dicho programa será el intento de obtener una respuesta.
- Desde un punto de vista lógico la respuesta a esa pregunta es “YES”, si la pregunta puede ser derivada aplicando “deducciones” del conjunto de reglas y hechos dados.

Ejecución de Programas: Ejemplo

Programa que describe una relación binaria (rel) y su cierre (clos):

rel(a,b).
rel(a,c).
rel(b,f).
rel(f,g).
clos(X,Y) :- rel(X, Y).
clos(X,Y) :- rel(X, Z), clos(Z, Y).

?-clos(a,f) → **QUERY**



Programación Funcional

Paradigma Aplicativo o Funcional

Basado en el uso de funciones. Muy popular en la resolución de problemas de inteligencia artificial, matemática, lógica, procesamiento paralelo

Ventajas:

- Vista uniforme de programa y función
- Tratamiento de funciones como datos
- Liberación de efectos colaterales
- Manejo automático de memoria

Desventaja:

- Suelen ser interpretados por lo que pueden presentar cierta ineficiencia de ejecución.

Paradigma Funcional

Características de los lenguajes funcionales

- Provee un conjunto de funciones primitivas
- Provee un conjunto de formas funcionales
- Semántica basada en valores
- Transparencia referencial
- Regla de mapeo basada en combinación o composición
- Las funciones de primer orden

Paradigma Funcional

Funciones

- El VALOR más importante en la programación funcional es el de una FUNCIÓN
- Matemáticamente una función es un correspondencia :
 $f: A \rightarrow B$
- A cada elemento de A le corresponde un único elemento en B
- $f(x)$ denota el resultado de la aplicación de f a x
- Las funciones son tratadas como valores, pueden ser pasadas como parámetros, retornar resultados, etc.

Paradigma Funcional

Definiendo Funciones:

- Se debe distinguir entre el VALOR y la DEFINICIÓN de una función.
- Existen muchas maneras de DEFINIR una misma función, pero siempre dará el mismo valor, ejemplo:

DOBLE $X = X + X$

DOBLE' $X = 2 * X$

Denotan la misma función pero son dos formas distintas de definirlas

Tipo de una función

- Puede estar definida explícitamente dentro del SCRIPT, por ejemplo:

cuadrado::num $\rightarrow$ num (define el tipo)

cuadrado $x = x * x$ (definición de la función)

- O puede **deducirse/inferirse** el tipo de una función

Paradigma Funcional

Expresiones y valores

- La expresión es la noción central de la programación Funcional
- Característica más importante:
 - “Una expresión es su VALOR”
- El valor de una expresión depende ÚNICAMENTE de los valores de las sub expresiones que la componen.
- Las expresiones también pueden contener VARIABLES, (valores desconocidos)

Paradigma Funcional

Expresiones y valores

- La noción de Variable es la de “variable matemática”, no la de celda de memoria.
- Las expresiones cumplen con la propiedad de

“TRANSPARENCIA REFERENCIAL”: Dos expresiones sintácticamente iguales darán el mismo valor.
- **No** existen EFECTOS LATERALES”

Paradigma Funcional

- Ejemplos de expresiones para evaluar

Expresión	Valor	
47	47	
(* 4 7)	28	Se está utilizando una Función primitiva
(+ 49 5)	54	

- Definiendo funciones....

cuadrado $x = x * x$

min $x \ y = x$, if $x < y$
 y , if $x > y$

cube $(x) = x * x * x$

Paradigma Funcional

Un script es una lista de definiciones y

- Pueden someterse a evaluación. Ejemplos:

?cuadrado (3 + 4)

49

?min 3 4

3

- Pueden combinarse, Ejemplo:

?min(cuadrado (1 + 1) 3)

3

- Pueden modificarse, ejemplo: Al script anterior le agrego nuevas definiciones:

lado = 12

area = cuadrado lado

Paradigma Funcional

- Algunas expresiones pueden **NO** llegar a reducirse del todo, ejemplo: $1/0$
- A esas expresiones se las denominan **CANÓNICAS**, pero se les asigna un **VALOR INDEFINIDO** y corresponde al símbolo $\text{bottom}(\wedge)$
- Por lo tanto toda **EXPRESIÓN** siempre denota un **VALOR**

Paradigma Funcional

Evaluación de las expresiones:

- La forma de evaluar es a través de un mecanismo de REDUCCIÓN o SIMPLIFICACIÓN
- Ejemplo:

cuadrado (3 + 4)

=> cuadrado 7

se aplicó (+)

=> 7 * 7 (cuadrado)

se aplicó cuadrado

=> 49

se aplicó cuadrado(*)

Otra forma sería:

cuadrado (3 + 4)

=> (3 + 4) * (3 + 4) (cuadrado)

=> 7 * (3 + 4) (+)

=> 7 * 7 (+)

=> 49 (*)

“No importa la forma de evaluarla, siempre el resultado final será el mismo”

Paradigma Funcional

Existen dos formas de reducción:

- Orden aplicativo

Aunque no lo necesite
SIEMPRE evalúa los
argumentos

- Orden normal (lazy evaluation)

- Diferida (Haskell)

No calcula más de lo necesario
La expresión **NO** es evaluada hasta que
su valor se necesite
Una expresión compartida **NO** es
evaluada más de una vez

Paradigma Funcional

TIPOS {
 Básicos
 Derivados

Básicos: Son los primitivos, ejemplo:
 NUM (INT y FLOAT) (Números)
 BOOL(Valores de verdad)
 CHAR(Characteres)

Derivados: Se construyen de otros tipos, ejemplo:
 (num,char) Tipo de pares de valores
 (num→→char) Tipo de una función

TODA FUNCIÓN TIENE ASOCIADO UN TIPO

Paradigma Funcional

- Expresiones de tipo polimórficas:

En algunas funciones no es tan fácil deducir su tipo.

Ejemplo:

id x = x

Esta función es la **función Identidad**

Su tipo puede ser de $\text{char} \rightarrow \text{char}$, de $\text{num} \rightarrow \text{num}$, etc.

Por lo tanto su tipo será de $\beta \rightarrow \beta$ (denota un tipo polimórfico)

Se utilizan **letras griegas** para representar **tipos polimórficos**

Otro ejemplo: **letra x = “A”**

Su tipo será $\beta \rightarrow \text{char}$

Paradigma Funcional

- Currificación:

- Mecanismo que reemplaza argumentos estructurados por argumentos más simples.
- Ejemplo: sean dos definiciones de la Función “Suma”

1. $\text{Suma}(x,y) = x + y$

2. $\text{Suma}'\ x\ y = x + y \quad \longrightarrow \quad \text{Suma}'\ x\ y = \text{Suma}'x\ (y) = x + y$

Existen entre estas dos definiciones una diferencia sutil:
“Diferencia de tipos de función”

El tipo de Suma es : $(\text{num}, \text{num}) \rightarrow \square \text{ num}$

El tipo de Suma' es : $\text{num} \rightarrow (\text{num} \rightarrow \square \text{ num}) \leftarrow$

Aplicando la función:

$$\text{Suma}(1,2) \rightarrow 3$$

$$\text{Suma}'\ 1\ 2 \square \quad \text{Suma}'1 \text{ aplicado al valor } 2 \rightarrow 3$$

Por cada valor de x devuelve una función

Para todo los valores devuelve el siguiente

Paradigma Funcional

Cálculo Lambda

- El un modelo de computación para definir funciones
- Se utiliza para entender los elementos de la programación funcional y la semántica subyacente, independientemente de los detalles sintácticos de un lenguaje de programación en particular.
- Es un modelo de programación funcional que se independiza de la sintaxis del lenguaje de programación



Paradigma Funcional

Las expresiones del Lambda cálculo pueden ser de 3 clases:

- Un simple identificador o una constante. Ej: x , 3
- Una definición de una función. Ej: $\lambda x.x+1$

Una aplicación de una función. La forma es $(e1\ e2)$, dónde se lee $e1$ se aplica a $e2$.

Ej: en la funcion cube $(x) = x * x * x$

$\lambda x. x * x * x$

$\lambda (x. x * x * x) 2$

Evaluamos la
función con 2
y resulta en 8

Enlaces

[Programación Lógica](#)

[Programación Funcional](#)

[Programación Funcional \(video\)](#)

Programación Orientada a Aspectos

Programación orientada a Aspectos

Divide responsabilidades en módulos conocidos como **aspectos**.

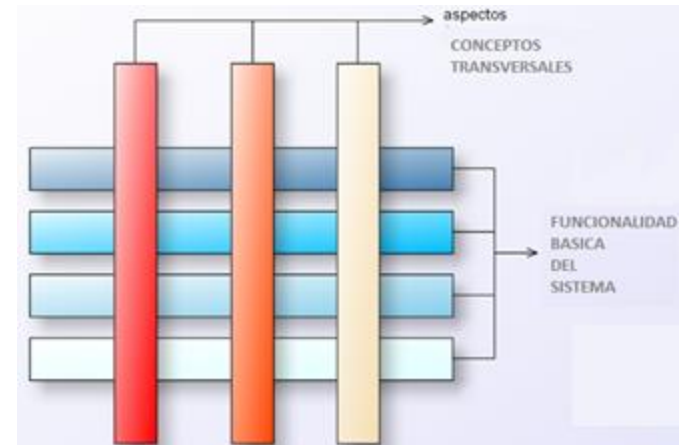
Estos aspectos suelen ser “transversales” a toda la aplicación.

Se puede combinar con otros paradigmas.

Su implementación está en un solo lugar en vez de repartirse en todo el código. (Mantenimiento y modificabilidad)

Usos:

- Autenticación
- Seguridad
- Trazabilidad
- Control de errores
- Otros.



Programación orientada a Aspectos

Conceptos

- Aspecto: Funcionalidad transversal de la aplicación (Ej. login, logs del sistema)
- Punto de enlace (joinpoint): Punto del programa donde el aspecto es conectado.
 - Llamada a un método
 - lanzamiento de excepción
 - Modificación de campo
 - Etc.
- Consejo (advice): Contiene el código que se ejecutará al implementar el Aspecto. Se inserta a través de los joinpoints.
- Punto de corte: define que advice (consejo) se usa en cada joinpoint.
- Introducción: Permite añadir métodos o atributos a una clase existente. Ej: en una auditoría podríamos establecer la fecha de modificación (como atributo) de un objeto.
- Target (objetivo): Clase sobre la que se aplica el aspecto.
- Proxy: Objeto resultante de aplicar un advice (consejo) a un objeto.
- Weaving (costura): integrar el advice (consejo) a los Target de nuestra aplicación.

Programación Orientada a Aspectos

Formas de definir cuando se ejecuta un aspecto.

- Antes
- Antes de retornar luego de la ejecución de un método.
- Después de una excepción
- Otras

Desventajas

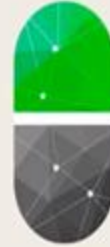
- Puede ser engorroso de seguir el flujo del programa (tanto en debug como al depurar)
- Dificultad de identificación de problemas.

Ventajas

- Mejora de modularidad
- Elimina dependencias
- Facilita la extensibilidad
- Reutilización de código
- Mejora el mantenimiento del código
- Código más limpio

Programación Orientada a Aspectos

AOP



```
public void insertarCliente (Cliente elCliente, String IdUsuario){
```

```
// código para login
```

```
// código para comprobación usuario válido
```

```
Session miSesion=sessionFactory.getCurrentSession();
```

```
miSesion.save (elCliente);
```

```
}
```

Controlador

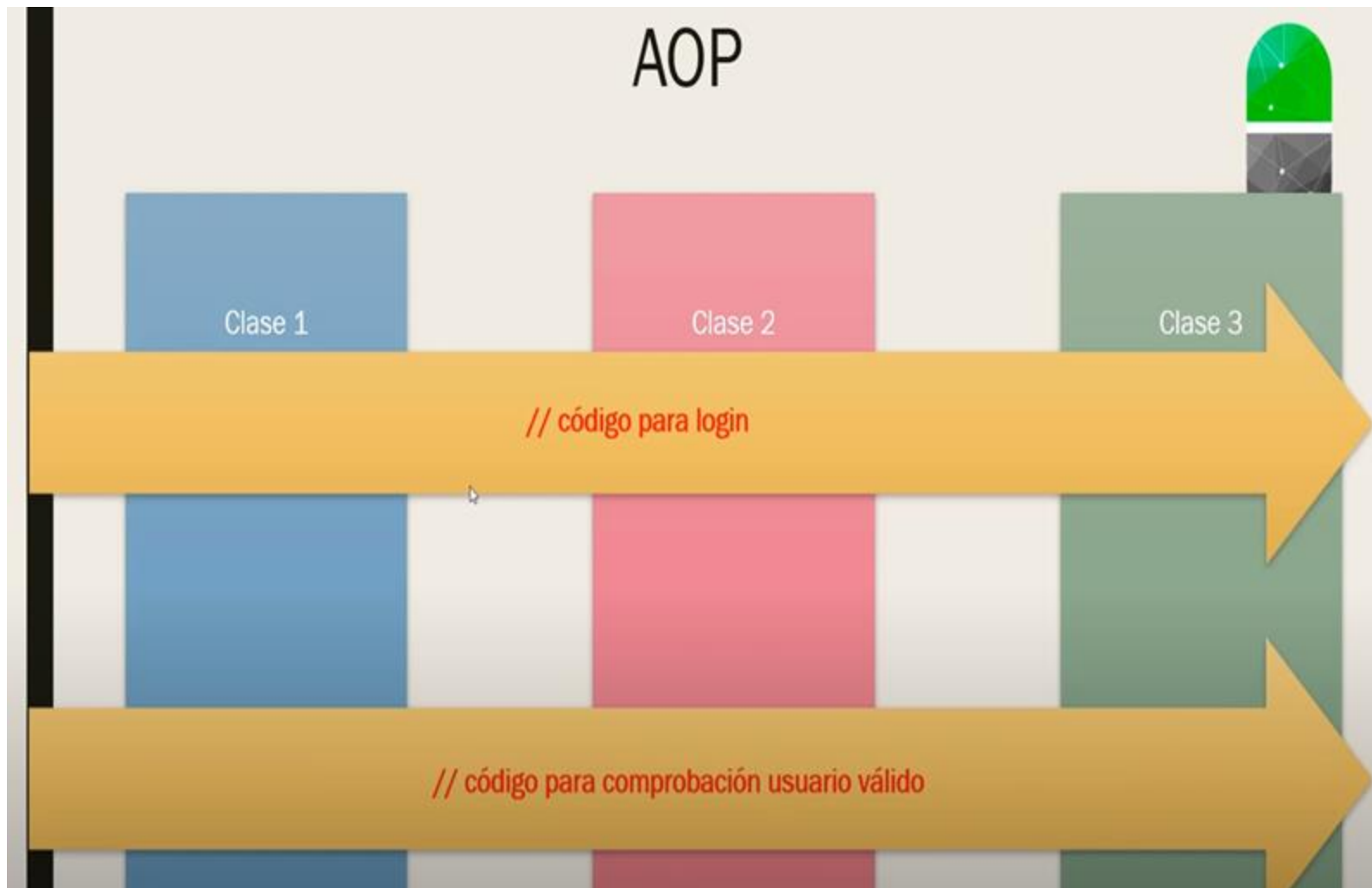
```
// código para login *  
// código para comprobación
```

ClienteDAO

```
// código para login  
// código para comprobación  
usuario válido
```



Programación orientada a Aspectos



Programación Orientada a Aspectos

```
public class Hello {  
  
    public static void main(String[] args) {  
        sayHello();  
    }  
  
    public static void sayHello() {  
        System.out.print("Hola ");  
    }  
}
```

```
public aspect AspectHello {  
  
    pointcut greeting() : execution (* Hello.sayHello(..));  
  
    before(): greeting() {  
        System.out.println("Bienvenidos a POA");  
    }  
  
    after() returning() : greeting() {  
        System.out.println("clase de lenguajes de programacion 2022-1");  
    }  
}
```

Bienvenidos a POA

Hola clase de lenguajes de programacion 2022-1

Process finished with exit code 0

Programación Orientada a Aspectos

Ejemplos de lenguajes con programación orientada a aspectos:

- Java: A través de frameworks como Spring AOP y AspectJ.
- C#: Con el framework PostSharp, entre otros.
- Python: Con librerías como Python AOP.
- Perl: Mediante el módulo Aspect.
- Ruby: Con librerías como Ruby AOP.